Búsqueda Inteligente de Objetos en Simulación

Estado actual del proyecto y plan de implementación
Detección (YOLOE) + Búsqueda (VLMaps) + Navegación (Habitat-Sim)
Extensión: ROS 1 Noetic + Toyota HSR

Mario García Blasco

13 de abril de 2026

Índice

1. Visión general del proyecto **4**
1.1. Problema . . . 4
1.2. Hipótesis . . . 4
1.3. Los tres niveles de conocimiento . . . 4
1.4. Idea clave: VLMaps estrecha la búsqueda, YOLOE confirma . . . 4
1.5. Etiquetado de habitaciones (LabelMe) . . . 5

2. Estado actual del proyecto (26 marzo 2026) **5**
2.1. Infraestructura — COMPLETADA . . . 5
2.2. Navegación interactiva — COMPLETADA . . . 5
2.3. Mapper LLM — COMPLETADO . . . 6
2.4. YOLOE como verificador — COMPLETADO . . . 6
2.5. Filtro de coherencia de heatmaps — COMPLETADO . . . 6
2.6. Filtro por tamaño medio de regiones — COMPLETADO . . . 6
2.7. Centrado visual con YOLOE — COMPLETADO . . . 7
2.8. Filtro de pared (wall filter) — COMPLETADO . . . 7
2.9. Room-based reasoning en Mapper LLM — COMPLETADO . . . 7
2.10. Colocación de objetos personalizados — COMPLETADO . . . 7
2.11. Lo que FALTA por implementar . . . 7

3. Arquitectura objetivo del sistema **8**
3.1. Flujo completo para una instrucción . . . 8
3.2. Flujo para objeto directo VLMap . . . 8

4. Los 5 métodos de búsqueda **9**
4.1. M1: Búsqueda aleatoria (baseline) . . . 9
4.2. M2: Candidato más cercano por BFS (implementado) . . . 9
4.3. M3: Búsqueda semántica (VLMap quality ranking) . . . 9
4.4. M4: Selección híbrida + navegación activa . . . 9
4.5. M5: Sistema completo . . . 9
4.6. Tabla resumen . . . 10

5. Componentes implementados en detalle **10**
5.1. Filtro de coherencia de heatmaps . . . 10
5.2. YOLOE en subproceso . . . 10
5.3. Scan multi-ángulo . . . 10

5.4. Detección y recuperación de colisiones . . . 11
5.5. Categorías directas, indirectas y YOLOE-incompatibles . . . 11
5.6. Filtro por tamaño medio de contornos . . . 11
5.7. Centrado visual (visual servoing) . . . 11
5.8. Filtro de pared (wall filter) . . . 12
5.9. Room-based reasoning . . . 12
5.10. Colocación de objetos personalizados . . . 12

6. Plan de implementación: trabajo futuro **13**
6.1. Paso 1 — Etiquetado de habitaciones (3 escenas) . . . 13
6.2. Paso 2 — Room-based reasoning en Mapper LLM . . . 13
6.3. Paso 3 — Heatmap region analysis completo . . . 13
6.4. Paso 4 — Implementar M1 y M3 baselines . . . 13
6.5. Paso 5 — Fórmula de scoring híbrida + M4 . . . 13
6.6. Paso 6 — M5: viewpoint planning + frontier exploration . . . 13
6.7. Paso 7 — Evaluación cuantitativa . . . 13
6.8. Paso 8 (Extensión) — Wrapper ROS 1 Noetic . . . 14

7. Orden de implementación **14**

8. Archivos clave del proyecto **14**

9. Tecnologías **14**

10. Preguntas frecuentes y decisiones de diseño **15**
10.1. P1: Categorías VLMap que YOLOE no reconoce (ej. “blinds”) . . . 15
10.2. P2: El path va por el otro lado de la pared . . . 16
10.3. P3: El robot navega demasiado cerca de paredes . . . 16
10.4. P4: El robot colisiona y se queda atascado . . . 16
10.5. P5: Cámara a –25° y scan multi-ángulo . . . 16
10.6. P6: Escenas MP3D más amplias . . . 17
10.7. P7: ¿Debería etiquetar las habitaciones? . . . 17
10.8. P8: ¿El robot usará la información de habitaciones? . . . 17
10.9. P9: ¿Puedo añadir objetos a la simulación para trials? . . . 18
10.10 P10: Visual centering (centrado del objeto en cámara) . . . 18
10.11 P11: Filtro de área media de contornos . . . 19

11. Integración HSSD: estado, cambios y guía de uso **19**
11.1. ¿Qué es HSSD y por qué se integra? . . . 19
11.2. Escena seleccionada . . . 19
11.3. Cambios realizados en el sistema . . . 20
11.4. Flujo completo con HSSD . . . 20
11.5. Anotaciones de habitaciones en HSSD (sin LabelMe) . . . 21
11.6. Diferencias respecto a MP3D . . . 22

12. Mejora de fluidez en navegación automática **22**
12.1. Método 1: reducir el tamaño de paso (step size) . . . 22
12.2. Método 2: viewer nativo de Habitat-sim . . . 23
12.3. Recomendación . . . 24

13. Correcciones del pipeline de navegación HSSD (31/03/2026) **24**
13.1. Problema 1: mapa de obstáculos invertido (suelo no detectado) . . . 24
13.2. Problema 2: heatmap ruidoso (falsos positivos de CLIP) . . . 24

13.3. Problema 3: robot atraviesa paredes (sin navmesh en HSSD) 25
13.4. Problema 4: verificación YOLOE al llegar al destino 25
13.5. Problema 5: camino planificado no visible en el mapa 25
13.6. Cambio adicional: cámara en primera persona con inclinación 26
13.7. Resumen de archivos modificados 26

1. Visión general del proyecto

1.1. Problema

Dado un objetivo expresado en lenguaje natural (por ejemplo, *"find the laptop"*), un robot simulado debe localizar el objeto de forma eficiente en un entorno interior. El robot opera en escenas Matterport3D y HSSD renderizadas con Habitat-Sim.

1.2. Hipótesis

Un sistema que combine tres niveles de conocimiento — etiquetas de habitaciones, memoria semántica de mobiliario (VLMaps, 37 categorías MP3D / 36 categorías HSSD) y verificación visual de vocabulario abierto (YOLOE) — encuentra objetos específicos de forma más eficiente que métodos que dependen de un solo nivel.

1.3. Los tres niveles de conocimiento

Nivel	Herramienta	Qué sabe
Habitación	LabelMe (MP3D) / `semantic_config.json` (HSSD)	«Esto es el salón»
Mueble / zona	VLMaps (37 cats MP3D / 36 cats HSSD)	«Hay un sofá allí»
Objeto específico	YOLOE (open-vocab)	«Hay un portátil encima de esa m

1.4. Idea clave: VLMaps estrecha la búsqueda, YOLOE confirma

1. El usuario pide un objeto específico («laptop»).
2. Un LLM traduce ese objeto a categorías VLMap donde es probable encontrarlo: `laptop` → `[table, counter, shelving]`.
3. VLMaps localiza todas las instancias de esas categorías en el mapa.
4. El robot navega al candidato más prometedor.
5. YOLOE mira con la cámara y confirma si el objeto está ahí.
6. Si no → siguiente candidato. Si sí → **ENCONTRADO**.

1.5. Etiquetado de habitaciones

Para instrucciones del tipo *"go to the living room and find the laptop"*, el robot necesita saber dónde está el salón.

- **MP3D**: etiquetado manual offline con LabelMe (pipeline lista, pendiente de ejecutar).
- **HSSD**: anotaciones nativas en `semantics/scenes/<id>.semantic_config.json`. La clase `SemanticSceneRoomProvider` las rasteriza sobre el grid del VLMap sin necesidad de LabelMe.

2. Estado actual del proyecto (13 abril 2026)

2.1. Infraestructura — COMPLETADA

Docker con CUDA 12.1 + Habitat-Sim 0.3.1 + conda (Python 3.9)

3 escenas MP3D con VLMaps construidos: **gTV8FGcVJC9_1**, **jh4fc5c5qoQ_1**, **mJXqzFtmKg4_1**

4 escenas adicionales descargadas (sin VLMap aún): mJXqzFtmKg4, jh4fc5c5qoQ, JeFG25nYj2p, Uxmj2M2itWa

1 escena HSSD integrada: **102344280** (15 habitaciones, ∼447 m^2), VLMap construido e indexado

Menú interactivo en Docker (`menu.sh`) para toda la pipeline

Submodule git `third_party/vlmaps` (rama `tfg-modifications`)

Repositorio limpio en GitHub: `mobileclip_blt.ts` (571 MB) eliminado del historial con `git-filter-repo`; push operativo

2.2. Navegación interactiva — COMPLETADA

Pipeline completa: instrucción → LLM → goal selection → visgraph → navegación live con visualización

`plan_path_only()`: planificación sin ejecución — elimina el teletransporte; camino mostrado como línea roja 800 ms antes de ejecutar

Movimientos lentos y fluidos (`_NAV_STEP_DELAY_MS = 80 ms`)

Dirección de cara corregida: $\theta = \arctan 2(-\Delta y, -\Delta x)$, signo de giro consistente con el planificador interno

Goal selection con 3 niveles de fallback (room centroid / BFS / heatmap peak)

Visgraph con margen de 4 px (∼20 cm); recuperación de colisión (giro 30° + avance + replanificación); hasta 3 replanes y 8 candidatos

Heatmap semántico overlay (JET colormap, 50 % opacidad)

Cámara 1ª persona con inclinación −30

Multi-goal retry (hasta 8 candidatos por categoría, ordenados por BFS)

2.3. Mapper LLM — COMPLETADO

El LLM traduce objetos no-VLMap a categorías VLMap probables. Ejemplo: `laptop` → `[table, counter, shelving]`

Distingue queries directas (`sofa` ∈ MP3D) e indirectas (`laptop` ∉ MP3D)

Room-based reasoning: sugiere habitaciones probables además de muebles. Ejemplo: `laptop` → `rooms: [office, bedroom]`

Prompt con 6 few-shot examples incluyendo rooms

2.4. YOLOE como verificador — COMPLETADO

Worker persistente (`_yoloe_persistent_worker.py`): modelo cargado una sola vez; inferencias ∼100–500 ms en lugar de ∼30 s

Protocolo stdout correcto: diagnósticos a `sys.stderr`, `start()` lee hasta recibir `READY"` (no solo una línea)

`mobileclip_blt.ts` copiado automáticamente al arrancar (busca en `parents[2]`, `parents[4]`, `/workspace`; elimina versión corrupta

$100MBparaforzarre-descarga)Capturadestderrdelworkerparadiagnsticodecrashes$

Centrado visual iterativo (visual servoing): hasta 6 iteraciones de 5° (±15 % tolerancia)

Flujo de verificación en 3 stages:

1. Standoff 0,25 m — YOLOE síncrono
2. Scan 360° in situ — YOLOE asíncrono (hilo bg)
3. Componente alternativa del heatmap — YOLOE síncrono

15 categorías YOLOE-incompatibles saltan la verificación (wall, floor, ceiling, beam, railing, etc.)

Caché global `get_session(target)`: reutiliza sesión entre instrucciones del mismo objeto

2.5. Filtros de calidad — COMPLETADOS

Filtro de coherencia de heatmaps: mín. 30 celdas, ≥30 % dominancia, ≤20 componentes conexas

Filtro por tamaño medio de regiones: descarta contornos <30 % del área media

Wall filter: ratio BFS/euclídea > 3× descarta candidatos al otro lado de paredes

Postprocess heatmap: suavizado Gaussiano → umbral relativo → metadatos por componente (área, score, densidad, centroide)

2.6. Integración HSSD — COMPLETADA

`dataset_type=hssd` activa la rama HSSD sin romper MP3D

`scene_dataset_config_file` declarado en Hydra config (sin prefijo `+`), corregido en `menu.sh`

Navmesh recomputado al iniciar con parámetros correctos

36 categorías HSSD en `get_categories("hssd")`

`SemanticSceneRoomProvider`: lee `semantic_config.json` de HSSD y rasteriza polígonos de habitación sobre el grid VLMap

Navegación interactiva validada en HSSD escena 102344280

2.7. Colocación de objetos personalizados — COMPLETADA

`place_objects.py`: define objetos 3D (.glb) via JSON config

Carga automática al iniciar; objetos visibles para YOLOE pero NO en VLMap

2.8. Bugs conocidos pendientes

- **IndexError boundary**: goal exactamente en el borde del obstacle map (`index 193 is out of bounds for size 193`). Fix: clamp en `plan_path_only()` antes de `plan_to()`
- **YOLOE conf_thresh bajo**: umbral actual 0.25; demasiado permisivo para verificación de llegada

2.9. Resumen de estado

Componente	Estado
Infraestructura Docker + Habitat + VLMaps	Hecho
Navegación interactiva M2 (BFS + colisiones)	Hecho
Mapper LLM (objeto → cats + rooms)	Hecho
YOLOE persistente + 360° scan	Hecho
Visual centering (servo YOLOE)	Hecho
Filtros de calidad (coherencia, área, pared)	Hecho
Integración HSSD completa	Hecho
Camino sin teletransporte	Hecho
Room-based reasoning (LLM)	Hecho
Colocación objetos personalizados	Hecho
Bug: IndexError boundary	Pendiente fix
Bug: conf_thresh YOLOE bajo	Pendiente fix
Heatmap region analysis (fórmula quality)	Pendiente
M1 baseline (aleatorio)	Pendiente
M3 baseline (VLMap quality ranking)	Pendiente
LLM iterativo con contexto de búsqueda	Pendiente
M4: fórmula híbrida + look-while-move	Pendiente
M5: viewpoints + frontier exploration	Pendiente
Evaluación cuantitativa automática	Pendiente
Wrapper ROS 1 Noetic (Toyota HSR)	Pendiente
Etiquetado LabelMe (MP3D, 3 escenas)	Pipeline lista
VLMaps JeFG25nYj2p, Uxmj2M2itWa	Escenas descargadas

3. Los 5 métodos de búsqueda

Todos comparten la misma infraestructura (visgraph, navegación, visualización). Lo que cambia es la **estrategia de selección de goal** y el comportamiento durante el tránsito.

3.1. M1: Búsqueda aleatoria (baseline)

- **Goal selection**: punto aleatorio navegable del mapa.
- **Verificación**: YOLOE al llegar.
- **Si no encuentra**: otro punto aleatorio.
- **Estado**: No implementado.

3.2. M2: Candidato más cercano por BFS (implementado)

- **Goal selection**: si el objeto está en VLMap, BFS al más cercano. Si no, mapper LLM → categorías relacionadas → BFS al mueble más cercano.
- **Verificación**: YOLOE al llegar (scan 0°, ±30°) para indirectos. VLMap para directos.
- **Si no encuentra**: siguiente candidato por distancia (hasta 8 candidatos).
- **Estado**: Implementado y funcional.

3.3. M3: Búsqueda semántica (VLMap quality ranking)

- **Goal selection**: candidatos ordenados por heatmap region quality (fórmula: $0{,}3 \cdot \text{area} + 0{,}4 \cdot \text{mean_score} + 0{,}3 \cdot \text{density}$).
- **Verificación**: YOLOE al llegar.
- **Estado**: No implementado (requiere Paso B).

3.4. M4: Selección híbrida + navegación activa

- **Goal selection**: fórmula híbrida $\text{score}(c) = \alpha \cdot \text{visual}(c) + \beta \cdot \text{semantic}(c) - \gamma \cdot \text{cost}(c)$.
- **Durante tránsito**: look-while-move + re-routing si detecta candidato mejor.
- **Estado**: No implementado.

3.5. M5: Sistema completo

- **Goal selection**: igual que M4 + viewpoint planning (4 poses alrededor) + frontier exploration.
- **Estado**: No implementado.

Método	Goal sel.	Look-move	Re-route	Viewpoints	Estado
M1	Random	No	No	No	Pendiente
M2	BFS dist	No	No	No	Hecho
M3	VLMap quality	No	No	No	Pendiente
M4	$\alpha v + \beta s - \gamma c$	Sí	Sí	Frontal	Pendiente
M5	$\alpha v + \beta s - \gamma c$	Sí	Sí	4 ángulos	Pendiente

4. Plan de implementación: próximos pasos

4.1. Paso A — Bugs inmediatos (prioridad alta, 1 sesión)

A1 — IndexError boundary (1 línea). En `plan_path_only()` de `habitat_lang_robot.py`, clamp antes de llamar a `self.nav.plan_to()`:

```
pos = [
    min(pos[0], self.nav.obs_map.shape[0] - 1),
    min(pos[1], self.nav.obs_map.shape[1] - 1),
]
```

No se modifica `navigation_utils.py` (código original de vlmaps). El clamp va en el código propio, antes de la llamada.

A2 — Umbral de confianza YOLOE. Subir `conf_thresh`:

- Stage 1 y 3 (verificación de llegada): **0.45** — si el robot está frente al objeto, debe verlo con alta confianza.
- Stage 2 (scan 360°): **0.30** — el objeto puede aparecer parcialmente o de lado durante el giro.

4.2. Paso B — Heatmap region analysis completo (1–2 sesiones)

Implementar la función de calidad por región del heatmap:

$$\text{quality}(r) = 0{,}3 \cdot \text{norm}(\text{area}) + 0{,}4 \cdot \text{mean_score} + 0{,}3 \cdot \text{density}$$

Donde:

- norm(area): área del componente normalizada respecto al mayor componente
- mean_score: puntuación CLIP media de los voxels del componente
- density: fracción de celdas activas dentro del bounding box del componente

Los metadatos por componente ya se calculan en `postprocess_heatmap()` (área, max, media, bbox, centroide, score). Solo hay que combinarlos en esta fórmula y exponerlos para que el selector de goal los use.

Esto es el prerequisito directo para M3.

4.3. Paso C — Baselines M1 y M3 (1 sesión)

M1 (aleatorio): seleccionar un punto navegable aleatorio del mapa como goal. Sirve como baseline mínimo para demostrar que VLMaps aporta valor.

M3 (semántico): en lugar de ir al candidato más cercano por BFS (M2), ir al candidato con mayor `quality` score (Paso B). Esto mide si la información semántica del heatmap mejora sobre la proximidad pura.

4.4. Paso D — Razonamiento semántico iterativo con LLM (1–2 sesiones)

Inspirado conceptualmente en MoMa-LLM (Honerkamp et al., RSS SemRob 2024). Implementación propia desde cero, sin uso del código del repositorio de Toyota Motor Europe.

Mejorar el Mapper LLM para incluir **contexto del estado actual de búsqueda**. En lugar de que el LLM decida categorías una sola vez al principio, recibe también:

- Qué habitaciones tiene la escena (de `SemanticSceneRoomProvider` para HSSD o LabelMe para MP3D)
- Qué candidatos se han visitado ya y cuál fue el resultado (fallado / confirmado por YOLOE)
- Qué habitaciones se han explorado

El LLM recibe este contexto estructurado y decide dónde buscar a continuación. Ejemplo:

```
Scene rooms: kitchen, living_room, bedroom_1, bedroom_2
Already searched:
  - table in living_room: YOLOE not found
  - counter in kitchen: YOLOE not found
Remaining candidates: shelving in office, table in bedroom_1
Where should the robot go next, and why?
```

Esto convierte M2 en un método adaptativo. Se implementa como extensión del Mapper LLM existente, añadiendo el historial de búsqueda como campo adicional en el prompt.

4.5. Paso E — Evaluación cuantitativa (1–2 sesiones)

Script automático que ejecuta N trials con objetos conocidos y mide:

- **Success Rate (SR)**: fracción de episodios donde el objeto se encuentra
- **Correct First Goal**: fracción donde el primer candidato visitado es correcto
- **Path Length (SPL)**: distancia total recorrida normalizada por el óptimo
- **Wrong Visits**: candidatos visitados antes del correcto
- **Time to Find**: pasos de simulación hasta encontrar

Output: CSV con resultados + gráficas comparativas M1 vs M2 vs M3 para la memoria del TFG.

Para HSSD: usar los objetos con anotaciones nativas del `semantic_config.json` para generar trials con ground truth automatizado.

4.6. Paso F — M4: fórmula híbrida + look-while-move (2–3 sesiones)

Goal selection con fórmula híbrida:

$$\text{score}(c) = \alpha \cdot \text{visual}(c) + \beta \cdot \text{semantic}(c) - \gamma \cdot \text{cost}(c)$$

Durante el tránsito: si el robot pasa cerca de un hotspot VLMap relevante, ejecuta una verificación YOLOE lateral sin detenerse. Si detecta el objeto, re-routing inmediato.

4.7. Paso G — M5: viewpoints + frontier exploration (3–4 sesiones)

Viewpoints: al llegar a un candidato, el robot se coloca en 4 puntos alrededor (a 1,5 m, cada 90°) y ejecuta YOLOE desde cada uno. Reduce falsos negativos por oclusión.

Frontiers: si se agotan todos los candidatos, explorar zonas inexploradas priorizando las cercanas a regiones VLMap relevantes.

4.8. Paso H — Wrapper ROS 1 Noetic para Toyota HSR (extensión)

- `vlmap_server` node: carga VLMap, expone servicio `/vlmap/query`
- `vlmap_nav` node: recibe instrucciones, parsea con LLM, envía goals a `move_base`
- Visualización en RViz: heatmap como `OccupancyGrid`, path como `nav_msgs/Path`

4.9. Orden de implementación

Prioridad	Paso	Depende de	Estado
—	M2 + YOLOE persistente	—	Hecho
—	HSSD integración completa	—	Hecho
1	A: Fix IndexError boundary + conf_thresh	—	Pendiente
2	B: Heatmap region analysis	—	Pendiente
3	C: M1 + M3 baselines	Paso B	Pendiente
4	D: LLM iterativo con contexto	—	Pendiente
5	E: Evaluación cuantitativa	Pasos C, D	Pendiente
6	F: M4 híbrido	Paso B	Pendiente
7	G: M5 viewpoints + frontiers	Paso F	Pendiente
8	H: Wrapper ROS Noetic	—	Pendiente
—	Etiquetado LabelMe (MP3D)	—	Pipeline lista
—	VLMaps JeFG25nYj2p, Uxmj2M2itWa	—	Pendiente

5. Referencia: MoMa-LLM

El paper *Language-Grounded Dynamic Scene Graphs for Interactive Object Search with Mobile Manipulation* (Honerkamp et al., RSS SemRob 2024) propone construir un scene graph dinámico mientras el robot explora y usar un LLM para razonar sobre él.

El código asociado (https://github.com/robot-learning-freiburg/MoMa-LLM) es propiedad de Toyota Motor Europe NV/SA y no puede usarse sin licencia expresa. Ninguna línea de ese código se utiliza en este TFG.

Las ideas conceptuales del paper (representación estructurada del estado para el LLM, room classification a partir de objetos observados, scoring semántico de fronteras) se usan como inspiración para el Paso D, con implementación completamente propia.

Diferencia principal: MoMa-LLM construye el mapa dinámicamente durante la exploración; este TFG usa un VLMap pre-construido, lo que permite evaluación más controlada y comparación directa entre métodos (M1–M5).

6. Archivos clave del proyecto

Archivo	Rol
`application/interactive_object_nav.py`	Pipeline de navegación principal
`vlmaps/robot/habitat_lang_robot.py`	Robot + visgraph + `plan_path_only`
`vlmaps/utils/llm_utils.py`	Mapper LLM: objeto → categorías + habitaciones
`vlmaps/utils/yoloe_utils.py`	YoloeSession: worker persistente, sync/async
`vlmaps/utils/_yoloe_persistent_worker.py`	Subprocess YOLOE aislado
`vlmaps/utils/room_provider.py`	SemanticSceneRoomProvider (HSSD nativo)
`vlmaps/utils/navigation_utils.py`	`plan_to_pos_v2`: visgraph + BFS
`config/object_goal_navigation_cfg.yaml`	Config Hydra principal
`docker/menu.sh`	Menú interactivo Docker

7. Historial de actualizaciones

31 de marzo de 2026 — Correcciones del pipeline HSSD

1. **Mapa de obstáculos invertido**: modelo de dos bandas (suelo $h < 0{,}15$ m / obstáculo $0{,}2 \leq h < 1{,}5$ m). Una celda navegable = 1 si tiene suelo Y no tiene obstáculo.
2. **Heatmap ruidoso**: argmax binario reemplazado por score CLIP continuo con umbral 0.3 + max-pooling + difusión solo desde celdas con score alto.
3. **Robot atraviesa paredes en HSSD**: `sim.recompute_navmesh()` en `_setup_sim()` cuando `dataset_type == "hssd"`.
4. **Integración YOLOE**: `yoloe_utils.py` + `_yoloe_worker.py`, superposición sobre ventana 1st person.
5. **Camino visible**: `path_cells` pasado a `show_map()`, color rojo.
6. **Cámara primera persona**: inclinación $-\pi/8$ rad ($\approx 22{,}5$).

2 de abril de 2026 (primera iteración)

1. **Postproceso de heatmaps**: suavizado Gaussiano → umbral relativo → metadatos por componente (área, max, media, bbox, centroide).
2. **Exploración 360°**: pasos de 20°, detención al detectar.
3. **Ruta alternativa**: componente con mayor score a >20 celdas.
4. **Movimiento fluido**: sin `set_agent_state()` visible.

2 de abril de 2026 (segunda iteración)

1. **Worker YOLOE persistente**: modelo cargado una sola vez; protocolo stdin/stdout; inferencias ∼100–500 ms.

2. **YoloeSession**: modos síncrono y asíncrono (hilo bg).
3. **Scan 360° fluido**: pasos nativos de 5°, YOLOE en hilo bg.
4. **Cámara** -30: `habitat_utils.py`.
5. **Standoff 0,25 m**: acercamiento máximo al objeto.

13 de abril de 2026

1. **Bug protocolo stdout/stderr YOLOE**: `_ensure_mobileclip()` imprimía en stdout interceptando el `READY"`; corregido a `sys.stderr`. `start()` ahora lee hasta recibir `READY"` (no solo una línea).
2. **mobileclip_blt.ts**: búsqueda en múltiples rutas candidatas; eliminación automática de versión corrupta (

$100MB$).**Captura stderr del worker** : $crashesvisiblesenconsola.$

3. **Dirección de cara definitiva**: $\arctan 2(-\Delta y, -\Delta x)$, signo consistente con el planificador.
4. **Sin teletransporte**: `plan_path_only()` + camino 800 ms antes de ejecutar.
5. **Hydra corregido**: `scene_dataset_config_file` sin prefijo `+` en ejecución y documentación de `menu.sh`.
6. **Repositorio GitHub limpio**: `mobileclip_blt.ts` (571 MB) eliminado del historial con `git-filter-repo`; push operativo.
7. **Bugs conocidos**: IndexError boundary + conf_thresh YOLOE bajo.